

Prueba técnica

Desarrollador Full Stack · Python (FastAPI) + Next.js

1. Objetivo

Construir una aplicación web sencilla que muestre un listado de productos en una tabla y permita ver el detalle de cada producto. El objetivo no es la cantidad de funcionalidades, sino la calidad del código, la claridad de la estructura, las pruebas y las decisiones técnicas que tomes. Prefiere una solución simple y bien terminada antes que una solución amplia e incompleta.

Sobre el uso de herramientas de IA: está permitido y damos por hecho que las usarás. Por lo mismo, lo que evaluamos no es que el código funcione, sino que entiendas lo que entregas y por qué. Habrá una sesión de revisión donde te pediremos explicar tus decisiones y hacer un cambio en vivo sobre tu propio código.

2. Stack requerido

Capa	Tecnología	Notas
Backend	Python 3.11+ con FastAPI	Pydantic para validación. ORM a elección (SQLAlchemy, SQLModel u otro).
Base de datos	A elección	SQLite es suficiente. También se acepta PostgreSQL con Docker.
Frontend	Next.js 14+ (App Router)	TypeScript recomendado. Estilos libres (Tailwind, CSS Modules, etc.).

3. Requisitos obligatorios

3.1 Modelo de producto

Como mínimo, cada producto debe tener los siguientes campos:

Campo	Tipo	Descripción
id	entero / UUID	Identificador único.
sku	texto	Código único del producto.
nombre	texto	Nombre comercial.
descripcion	texto	Descripción larga, opcional.
categoria	texto	Categoría o familia.
precio	decimal	Precio de venta.

Campo	Tipo	Descripción
stock	entero	Unidades disponibles.
imagen_url	texto	URL de la imagen, opcional.

3.2 Backend (FastAPI)

- **GET /products** — listado con paginación (*page*, *page_size*), búsqueda por texto sobre nombre o SKU, y filtros por categoría y por rango de precio.
- **GET /products/{id}** — detalle de un producto. Debe responder **404** si no existe.
- Validación de entrada y salida con esquemas de Pydantic.
- Manejo de errores con códigos HTTP correctos y mensajes claros.
- Documentación automática disponible en */docs*.
- Datos de prueba: incluir un script o *seed* con al menos 30 productos para poder revisar la tabla.

3.3 Frontend (Next.js)

- **Vista de listado:** tabla de productos que consume el backend y muestra, como mínimo, SKU, nombre, categoría, precio y stock.
- **Navegación al detalle:** al hacer clic en una fila se debe navegar a la vista de detalle del producto (por ejemplo, */products/{id}*). La fila completa debe ser navegable, no solo un botón.
- **Vista de detalle:** muestra todos los campos del producto y ofrece una forma clara de volver al listado.
- Estados de carga y de error visibles para el usuario (cargando, sin resultados, fallo de red).
- La aplicación debe ser usable en pantallas pequeñas; no se exige un diseño elaborado.

3.4 Pruebas unitarias

No pedimos una suite exhaustiva, pero sí queremos ver que sabes qué vale la pena probar. Como mínimo:

- Pruebas de la lógica de filtrado y paginación, **sin levantar la base de datos ni el servidor** (esa es la parte unitaria).
- Al menos una prueba de integración del endpoint de listado y otra del detalle, incluyendo el caso **404**.
- Al menos un caso borde real: búsqueda sin resultados, página fuera de rango, precio negativo o SKU duplicado.
- Indica en el README el comando exacto para ejecutarlas.

Una prueba que solo verifica *status_code == 200* no aporta información. Nos interesan las que fallarían si alguien rompe la lógica.

3.5 Entorno reproducible: local y con Docker

El entregable debe poder levantarse de **dos formas**, y ambas deben estar documentadas y funcionar tal como las describes:

- **Local:** backend y frontend ejecutándose directamente en la máquina, con sus dependencias declaradas (*requirements.txt*, *pyproject.toml*, *package.json*) y las versiones fijadas.
- **Dockerizado:** un *docker-compose up* desde la raíz debe levantar backend, frontend y base de datos, sin pasos manuales adicionales.
- Variables de entorno en un **.env.example** versionado, con valores por defecto que funcionen. Nunca subas credenciales reales al repositorio.
- Los datos de prueba deben cargarse igual en ambos modos (un comando o un paso del arranque).

Esto es un requisito, no un extra: si el proyecto no levanta siguiendo tu propio README en un equipo limpio, la evaluación se detiene ahí. Antes de entregar, prueba las instrucciones desde cero — idealmente clonando el repositorio en otra carpeta.

★ 4. Puntos extra (suman, no son obligatorios)

4.1 Patrón Criteria para los filtros de la tabla

Es el punto extra que más valoramos. En lugar de recibir los filtros sueltos como parámetros y armar la consulta dentro del endpoint, modela los criterios de búsqueda como un objeto propio del dominio: filtros (campo, operador, valor), orden y paginación. El endpoint traduce la petición HTTP a ese objeto, y una capa aparte lo traduce a la consulta concreta del ORM.

- Agregar un filtro nuevo no debería obligar a modificar el endpoint ni a concatenar SQL a mano.
- El objeto de criterios debe poder probarse de forma unitaria, aislado del ORM y de FastAPI.
- Define y valida los operadores que soportas (igual, contiene, mayor que, entre, etc.).
- Cuida la inyección de SQL y los campos no permitidos: no todo campo recibido debe ser filtrable.

4.2 Importar productos desde un archivo Excel

- Endpoint **POST /products/import** que reciba un archivo .xlsx y cargue los productos en la base de datos.
- Formulario de carga en el frontend con retroalimentación del resultado.
- Validar cada fila: campos obligatorios, tipos correctos y SKU duplicado.
- Responder un resumen del proceso: filas leídas, insertadas, actualizadas y rechazadas, indicando el número de fila y el motivo del rechazo.
- Que una fila inválida no invalide todo el archivo se considera un plus.

Formato esperado del archivo (primera fila con los encabezados):

sku	nombre	categoría	precio	stock	imagen_url
SKU-001	Taladro percutor 650W	Herramientas	89990	12	https://.../taladro.jpg
SKU-002	Casco de seguridad	Protección	12990	40	https://.../casco.jpg

4.3 Visualización de la imagen del producto

- Miniatura de la imagen en cada fila de la tabla.
- Imagen en tamaño grande en la vista de detalle.
- Imagen de reemplazo (*placeholder*) cuando el producto no tiene imagen o la URL falla.
- Uso del componente *next/image* con la configuración de dominios correspondiente.

4.4 Kardex del producto

En la vista de detalle, agrega el kardex del producto: el registro de sus movimientos de stock (entradas y salidas) con su efecto en el saldo. Define tú el modelo de datos y el método de valorización que utilizas, y justifica esa elección en el README.

- Cada movimiento debe registrar al menos: fecha, tipo, cantidad, costo unitario y documento de referencia.
- El saldo debe quedar expresado tanto en unidades como en valor.
- La lógica de cálculo del saldo debe estar cubierta por pruebas unitarias.

4.5 Otros aspectos valorados

- Paginación, búsqueda y ordenamiento resueltos en el servidor, no en el navegador.
- TypeScript con tipos derivados del contrato del backend.
- Imágenes Docker cuidadas: *multi-stage build*, sin dependencias de desarrollo en la imagen final.

5. Comprensión del negocio

Trabajamos con catálogos de productos industriales y con los equipos comerciales y de abastecimiento de nuestros clientes. Nos interesa que hablemos el mismo lenguaje de negocio, no solo el mismo lenguaje técnico. Responde estas preguntas **con tus palabras** en una sección del README. Son respuestas breves, de tres a cinco líneas cada una:

- 1 ¿Qué es un SKU? ¿En qué se diferencia del nombre del producto y de un código de barras, y por qué debe ser único?
- 2 ¿Para qué sirve un kardex y qué relación tiene con la valorización del stock?
- 3 Si un mismo producto se compró en distintas fechas a distintos costos, ¿qué costo usarías para valorizar el saldo y por qué?
- 4 ¿Qué diferencia hay entre el stock físico y el stock disponible para la venta? ¿Cómo lo reflejarías en el modelo de datos?

No buscamos definiciones de manual ni respuestas largas. Buscamos criterio: si no conoces un concepto, dilo y explica cómo lo resolverías. Una respuesta honesta vale más que una definición copiada.

6. Entrega

- Repositorio Git público (o archivo comprimido) con dos carpetas: **backend/** y **frontend/**.
- Archivo **README.md** con: requisitos previos, **los dos modos de ejecución (local y Docker)**, variables de entorno, cómo cargar los datos de prueba y el comando para ejecutar las pruebas.
- En el mismo README: las decisiones técnicas que tomaste, lo que dejaste fuera, qué mejorarías con más tiempo y las respuestas de la sección 5.
- Historial de *commits* con mensajes descriptivos; se valora más que un único commit final.
- **Tiempo estimado:** entre 4 y 6 horas para los requisitos obligatorios. No es necesario completar los puntos extra: elige los que mejor muestren tu forma de trabajar.

7. Criterios de evaluación

Criterio	Peso	Qué se observa
Calidad del código y diseño	25%	Organización, nombres claros, separación de responsabilidades, ausencia de duplicación. Aquí pesa el patrón Criteria si lo implementas.
Funcionalidad	20%	El listado y el detalle funcionan de extremo a extremo, sin errores en consola.
Pruebas	15%	Qué se decidió probar y si las pruebas detectarían una regresión real.
Comprensión del negocio	15%	Respuestas de la sección 5 y coherencia del modelo de datos con ellas.
Reproducibilidad	10%	El proyecto levanta local y con Docker siguiendo solo el README.

Criterio	Peso	Qué se observa
Diseño de la API	8%	Rutas coherentes, esquemas bien definidos, códigos HTTP correctos.
Frontend	7%	Manejo de estados, navegación, legibilidad de los componentes.

<> 8. Anexo: ejemplos de respuesta

GET /products?page=1&page_size=2&q=taladro&categoria=Herramientas

```
{
  "items": [
    {
      "id": 1,
      "sku": "SKU-001",
      "nombre": "Taladro percutor 650W",
      "categoria": "Herramientas",
      "precio": 89990.0,
      "stock": 12,
      "imagen_url": "https://ejemplo.cl/taladro.jpg"
    }
  ],
  "total": 1,
  "page": 1,
  "page_size": 2
}
```

POST /products/import (respuesta sugerida)

```
{
  "leidas": 50,
  "insertadas": 47,
  "actualizadas": 1,
  "rechazadas": 2,
  "errores": [
    {"fila": 14, "motivo": "precio no es un numero"},
    {"fila": 29, "motivo": "sku duplicado"}
  ]
}
```

Si alguna parte del enunciado te parece ambigua, resuélvela con el criterio que consideres correcto y déjalo anotado en el README. Esa decisión también forma parte de la evaluación.